FinFy Setup & Deployment Architecture

Local Environment Configuration & Audio Server Setup Guide

Now that you have exported and downloaded the full-stack `FinFy` codebase ZIP archive from Google AI Studio, the next phase is preparing your local workstation, organizing your raw media resources, configuring dependencies, and starting up the execution runtime loops.

> **Workspace Prerequisite**
>
> Ensure you have a terminal environment open and that **Node.js** (v18 or higher) is fully installed on your platform.

Step 1: Extract and Initialize Workspace Architecture

Unpack the compilation archive and inspect the structural hierarchy of your full-stack layout tree. Open your system terminal core and navigate cleanly into the target directory roots:

```
# Extract the source archive cluster
unzip FinFy-Music-App.zip
cd FinFy-Music-App

# Open the directory architecture directly inside Visual Studio Code
code .
```

Your workspace structural blueprint will map to the following component organization layers:

- `/backend` or `server.js` : Handles internal node clusters, routing targets, and HTTP range streaming streams.
- `/src` or `/frontend` : Houses the black and neon-orange UI canvas layers, reactive states, and tabs.
- `DESIGN.md` : Structural ruleset retaining all color token maps (`#000000` / `#FF6B00`).

Step 2: Populate the Media Repositories with Tracks

Your streaming backend requires raw audio payloads to stream out to your device environment layouts. Setup a static storage directory layout block inside your backend structural hierarchy:

```
# Navigate to your backend cluster workspace and establish the song mount
mkdir -p backend/songs
```

Drop a selection of operational `.mp3` data track segments directly inside this newly formed folder footprint. Name them clearly without irregular character configurations (e.g., `song1.mp3`, `track2.mp3`).

Step 3: Deploy Runtime Package Dependencies

Both frontend view wrappers and the processing backend need their modular packages pulled down from package registries before execution. Fire up these install chains inside your project terminal nodes:

```
# Install infrastructure dependencies inside the backend system environment
cd backend
npm install express cors dotenv

# Return and establish dependencies for the interactive user UI layer
cd ../frontend
npm install
```

Step 4: Execute the Audio Server Array

To let the application successfully populate data elements across your tab arrays and stream media pipelines, launch the concurrent execution streams:

A **Start the Node Streaming Server Core:**

```
cd backend
node server.js
```

The backend module will boot up on port configuration `http://localhost:5000`, awaiting HTTP requests.

B **Boot Up the Frontend Layout Engine:**

```
cd ../frontend
npm run dev
```

This pipeline creates a local web socket loop (typically mapping to address grid `http://localhost:5173`).

Step 5: Simulating and Testing on iOS Form Factors

Open your local application address inside your desktop browser runtime. To align layout items to match standard iOS specifications:

1. Press `F12` or right-click anywhere on the viewport canvas grid and click **Inspect Element**.
2. Locate and engage the **Device Emulation Frame Bar** (mobile device toggle graphic tool).
3. Change the device frame scale targets to a responsive **iPhone** canvas variant.
4. Interact with track listings to verify fluid progress bars and operational navigation layouts across your three active menu segments!